

Maximizing Math-Reasoning Accuracy in a 4B Thinking Model

How far can I push a fixed four-billion-parameter reasoning model with prompting and decoding alone, no training, under a single 12 GB GPU, and a high-precision grader?

Joseph Calderon

Individual submission · vLLM · RTX 4070 12 GB

0.724

FINAL LEADERBOARD, BEST OF TWO

The results, TL;DR

While the Qwen3-4B-Thinking-2507 was mandated by the rules, I was able to empirically uncover an optimum configuration with no training. Treating inference as an optimization problem, I raised the score from a 0.388 baseline to 0.724 without zero training. The gains came from finding an optimal token limit and thinking budget, and from aggressive prompting to write answers in the exact format expected by the grader.

- ▶ **Team:** An individual submission: pipeline, experiments, and error analysis by Joseph Calderon.
- ▶ **Approach:** Four inference-time levers: thinking-token budget, FP8 weights, a grader-aligned prompt, and light sampling.
- ▶ **Insight:** Uncovering when to stop thinking, how to format the answer, and an empirical search for the optimal prompt.

0.724

FINAL SCORE · BEST OF TWO RUNS

+0.336

GAIN OVER THE 0.388 BASELINE

0

PARAMETERS TRAINED

Three levers that did the work

Thinking-Budget Decoding

Bound the model's reasoning to a fixed token count, then feed it back in so the model expends a fixed number of tokens for reasoning and answering.

Grader-Aligned Prompt

Write every answer in the grader's own plain-ASCII notation, precision, and bracketing, so correct reasoning is rightfully rewarded.

Compute Budget

The token ceiling is a tradeoff: more reasoning helps increase accuracy but inflates runtime on my GPU.

■ SECTION ONE

Introduction

The problem, the rules, and constraints.

01

The task and team contributions

The assignment fixed the model but allowed fine-tuning and reinforcement learning. However, I wanted to challenge myself to push a four-billion-parameter thinking model as far as it will go using prompting and decoding only.

- ▶ No external data, tools, APIs, or calculators at inference. Every weight in the model had to originate from HuggingFace.
- ▶ This project was a solo effort. I built the pipeline, ran every experiment, and maintained a detailed record of the project history to find the best configuration.

1

PERSON TEAM

12 GB VRAM

ONE RTX 4070, UNDER WSL₂

■ SECTION TWO

Methodology

The benchmark, the model, and the levers available without training.

02

The dataset

- ▶ **The problems:** Mathematical reasoning from high-school to graduate level across many domains, delivered as JSONL, one problem per line.
- ▶ **Two formats:** Free-form items want an exact value for each [ANS] placeholder, and every sub-answer must be right; multiple choice wants a single capital letter.
- ▶ **Public vs private:** 1,126 public problems carry ground-truth answers for development; 943 private problems, answers withheld, decide the leaderboard on a similar distribution.

943

PRIVATE · GRADED

1,126

PUBLIC · DEV

INPUT PROCESSING: NO AUGMENTATION

No external data, synthetic examples, or few-shot exemplars. Every problem is used as written, and the only transformation is prompt assembly.

The model and my configuration

- ▶ **The model:** Qwen3-4B-Thinking-2507, required by the rules, served in its official FP8 build through vLLM.
- ▶ **How it answers:** A reasoning model: it writes a hidden chain of thought, then commits to a boxed final answer that the grader extracts.
- ▶ **Quantization:** Official FP8 outscored in-flight INT₄ by roughly a point and ran about 1.7 times faster, buying back the runtime that inflated from my increased token budget spend.
- ▶ **Another reason not to train:** Supervised fine-tuning was attempted and dropped as it collapsed accuracy from 57.5% to 15. The model was kept intact despite the lack of <think/> tags in the reasoning trace and pushed at inference.

Engineering tricks: my four levers

01 Two-phase thinking budget

Cap the hidden reasoning at a chosen token count, then feed it back to force a boxed answer. Tuned in Experiment 1.

02 FP8 over INT₄

The official FP8 build added about a point of accuracy and ran roughly 1.7 times faster, easing the runtime.

03 Grader-aligned prompting

State the answer in the grader's exact ASCII, precision, and brackets. The decisive lever, developed in Experiment 2.

04 Decoding at temperature 0.6

A light touch of sampling beat greedy decoding, and the gain compounded with the prompt. Tested head to head in Experiment 2.

■ SECTION THREE

Experiments

What was tested, and what each decision was worth.

03

Choosing a budget under a 12 GB ceiling

- ▶ **The trade-off:** Since a thinking model spends tokens before it answers, too tight a budget cuts reasoning mid-step, while too loose swells runtimes to hours on a 12 GB card.
- ▶ **The sweep:** An 8,192 token limit peaked at a 6,000 budget, and lifting the ceiling to 12,288 over 9,000 performed even better. 16,384 over 12,000 regressed, so 12,288 over 9,000 was our optimum.
- ▶ **The bottleneck:** Truncation harmed free-form far more than multiple choice, so the looser budget shields it. Even then, the formatting stayed the dominant error, which led us to Experiment 2.

Limit	Budget	Dev acc
8,192	6,000	63.7%
12,288	9,000	68.8%
16,384	12,000	67.7%

Overall accuracy, 80-question dev set.

WHEN FORCED TO ANSWER AT THE BUDGET:

81%

MULTIPLE CHOICE CORRECT

30%

FREE-FORM CORRECT

Engineering the prompt to the grader

Error analysis put the residual losses in free-form formatting, not reasoning. The reference answers are plain ASCII, while the model defaulted to LaTeX.

- 1

Variant B, the base
Structured reasoning, one boxed answer. 57.5% on dev, free-form lagging multiple choice.
- 2

Variants D and F
Format discipline: honor the requested precision, drop parenthesis-wrapped lists. F reached 70.0%.
- 3

Variant G, grader-matched
Plain ASCII over LaTeX, parentheses kept on pairs and intervals, 10+ significant figures. 0.703.
- 4

Variant I, the final
Answer-first multiple choice, 12 significant figures, a recount of the [ANS] parts. 0.724.

	Greedy	Temp 0.6
Prompt G	0.696	0.703
Prompt I	0.706	0.724

Leaderboard score, all 943 private questions.

TWO LEVERS THAT COMPOUND

Prompt I beat G at both settings, and temperature 0.6 beat greedy for both prompts. The two gains compound to +0.028.

■ SECTION FOUR

Discussion

What held up, what limited it, and where it leads.

04

Strengths, limits, next steps

STRENGTHS

- ▶ An accuracy gain of 33.6% over the baseline with zero training.
- ▶ A reproducible allowing cloners to run inference on any private set of their choosing so long as it is formatted correctly.
- ▶ Model configuration decisions were grounded in empirical error analysis.

LIMITS

- ▶ About five hours for one full run on my 12 GB GPU.
- ▶ Temperature 0.6 introduces variability that trades away exact reproducibility.
- ▶ A 4B-parameter model sets a hard ceiling on accuracy.

IMPROVEMENTS

- ▶ Self-consistency voting across sampled traces.
- ▶ Distilling the two-phase budget into one pass.

What I learned

“

This challenge was always an optimization task balancing thinking budgets and runtime with my hardware constraints. While there may have been more attractive alternatives, real value can always be extracted from empirical error analysis and relentless A/B testing.

TL;DR: Error analysis drove the work. 80% of free-form misses were due format errors and truncation, so formatting and budget got the effort as opposed to a bigger model. Net increase from 0.388 to 0.724 with zero training, while fine-tuning only hurt (57.5% to 15%).

Where this goes next

01

Vote

Combine several sampled reasoning traces through self-consistency.

02

Normalize

Train an answer-normalizer to close the residual plain-ASCII gaps that still cost free-form points.

03

Scale & distill

Carry the recipe to larger thinking models and distill the two-phase budget into a single pass.